

Chuyên đề Đệ Quy và Quy Hoạch Động

ThS. Nguyễn Hải Đăng

Định nghĩa

- **Đệ quy:** Một kỹ thuật gọi hàm định nghĩa chính nó.
 - Đệ quy thường được sử dụng để giải quyết các bài toán mang tính liệt kê hoặc xét toàn bộ không gian tìm kiếm.
- **Quy hoạch động:** Một phương pháp giải các bài toán có thể sử dụng đệ quy theo hướng *khử đệ quy* nhằm giảm bớt việc tính toán trùng lặp trong đệ quy.
 - Quy hoạch động thường được sử dụng để giải quyết các bài toán đếm số lượng, tìm cực trị.

Cách nhận diện bài toán

Đề quy:

- Các bài toán có yêu cầu **liệt kê** các trường hợp.
- **Vét cạn** / xét hết các trường hợp để tìm đáp án.

Quy hoạch động:

- Các bài toán **đếm / tổ hợp**.
- Các bài toán **tìm cực trị**, phương án tối ưu.

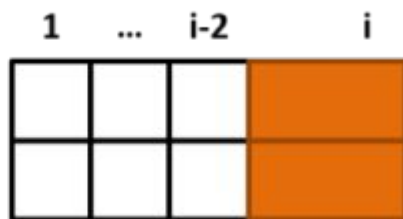
Bài toán mẫu - Lát gạch

Cho một nền nhà hình chữ nhật kích thước $2 \times N$. Hãy đếm số cách lát các viên gạch có kích thước 1×2 và 2×1 vào nền nhà trên sao cho toàn bộ nền nhà đều được lát gạch và không có phần nào của các viên gạch nhỏ thừa ra ngoài.

Bài toán mẫu - Lát gạch

Cho một nền nhà hình chữ nhật kích thước $2 \times N$. Hãy đếm số cách lát các viên gạch có kích thước 1×2 và 2×1 vào nền nhà trên sao cho toàn bộ nền nhà đều được lát gạch và không có phần nào của các viên gạch nhỏ thừa ra ngoài.

Gọi $F[i]$ là số cách lát các viên gạch nhỏ vào hình chữ nhật có kích thước $2 \times i$.

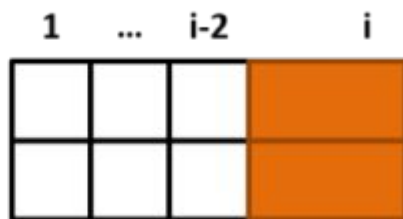


Bài toán mẫu - Lát gạch

Cho một nền nhà hình chữ nhật kích thước $2 \times N$. Hãy đếm số cách lát các viên gạch có kích thước 1×2 và 2×1 vào nền nhà trên sao cho toàn bộ nền nhà đều được lát gạch và không có phần nào của các viên gạch nhỏ thừa ra ngoài.

Gọi $F[i]$ là số cách lát các viên gạch nhỏ vào hình chữ nhật có kích thước $2 \times i$.

- Nếu dùng viên 2×1 thì $F[i] = F[i-1]$
- Nếu dùng viên 1×2 thì $F[i] = F[i-2]$
- $\Rightarrow F[i] = F[i-1] + F[i-2] \Rightarrow$ Bài toán fibonacci



Bài toán mẫu - Lát gạch

- **Dãy số Fibonacci được định nghĩa như sau:**
 - $\text{Fibo}(0) = 0$
 - $\text{Fibo}(1) = 1$
 - $\text{Fibo}(n) = \text{Fibo}(n-1) + \text{Fibo}(n-2)$
- **Dãy Fibonacci:**
 - 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...
- **Bài toán:** cho số nguyên dương n , tìm số Fibonacci thứ n . $\text{Fibo}(n) = ?$
- **Ví dụ:**
 - Input: $n = 10$
 - Output: $\text{Fibo}(10) = 55$

Các hướng tiếp cận và phân tích bài toán

- Gồm 2 hướng tiếp cận chính:
 - Top-down: góc nhìn theo hướng đệ quy
 - Bottom-up: góc nhìn theo hướng quy hoạch động

Các hướng tiếp cận và phân tích bài toán

- Top-down:
 - Chia nhỏ trường hợp lớn ra thành các trường hợp con
 - Tìm lời giải cho các trường hợp con để ra được kết quả cho bài toán
- Đây là hướng tiếp cận tương đối phức tạp vì thiếu sự chắc chắn.
- Tuy nhiên, trong một số bài toán, đây là cách tiếp cận dễ nhất để nhìn ra hướng giải quyết.
- Mấu chốt: Rã bài toán lớn thành các bài toán con để tìm lời giải.
- *Lưu ý các trường hợp trùng nhau.*

Các hướng tiếp cận và phân tích bài toán

- Bottom-up:
 - Giải quyết bài toán với *trường hợp nhỏ nhất* (ví dụ dãy với 1 phần tử)
 - Dùng kết quả của trường hợp nhỏ để tìm lời giải cho các trường hợp lớn hơn (dãy với 2 phần tử, 3 phần tử, ...)
- Đây là hướng tiếp cận trực quan và chắc chắn.
- Mấu chốt: tiếp cận và xây dựng lời giải cho trường hợp mới bằng cách sử dụng *kết quả của các trường hợp đã được tính sẵn* trước đó.

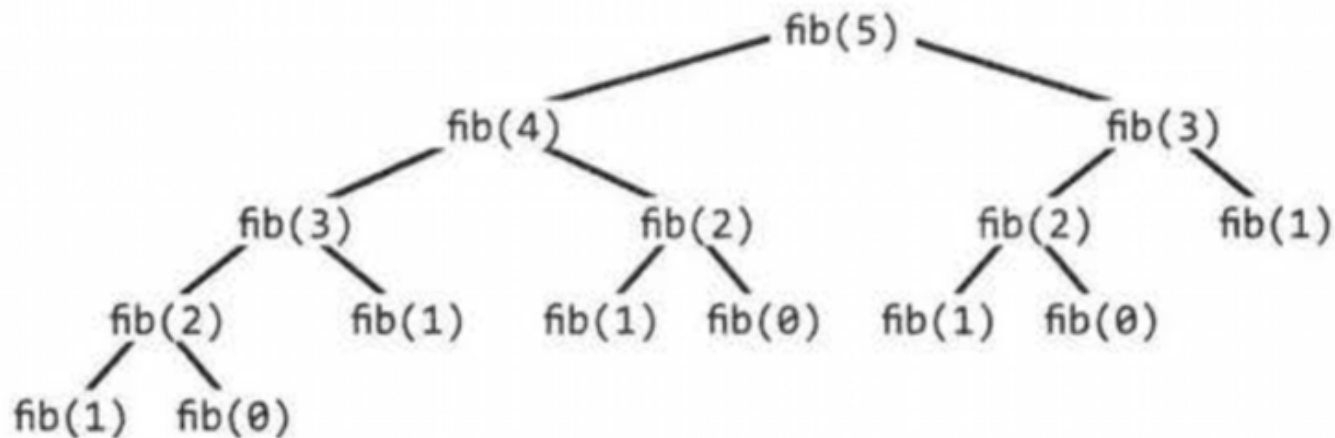
Bài toán Fibonacci với Đệ quy (góc nhìn top-down)

```
int fibo(int n) {  
    if (n==0 || n==1) return n;  
    return fibo(n-1) + fibo(n-2);  
}  
fibo_n = fibo(n);
```

- Độ phức tạp thuật toán:
 - Về thời gian: $O(n)$ hay $O(n^2)$ hay lớn hơn?
 - Về không gian: $O(n)$ or $O(\log_2(n))$?

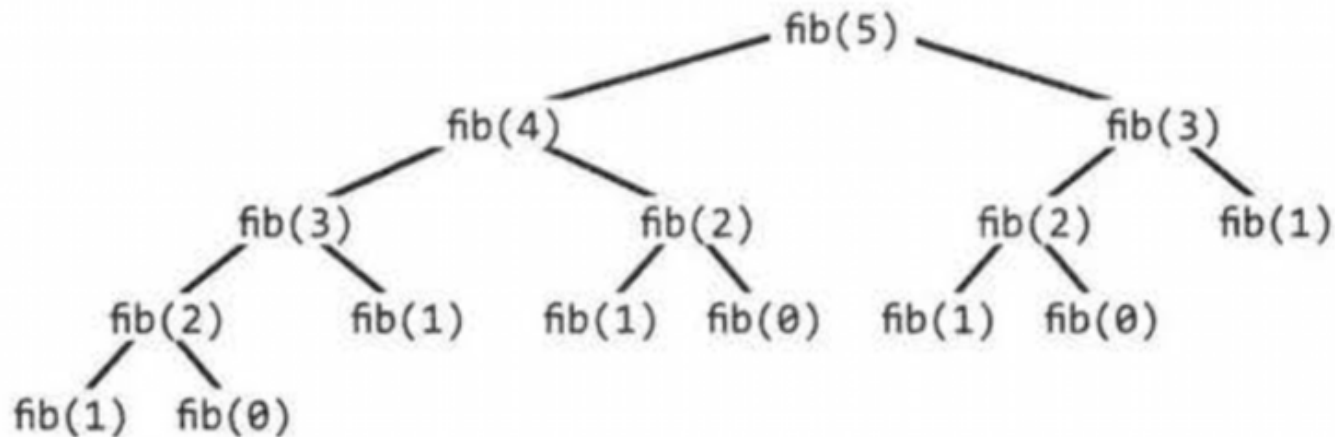
Bài toán Fibonacci với Đệ quy (góc nhìn top-down)

- Gợi ý: Nên sử dụng cây để biểu diễn và phân tích độ phức tạp của một thuật toán đệ quy.



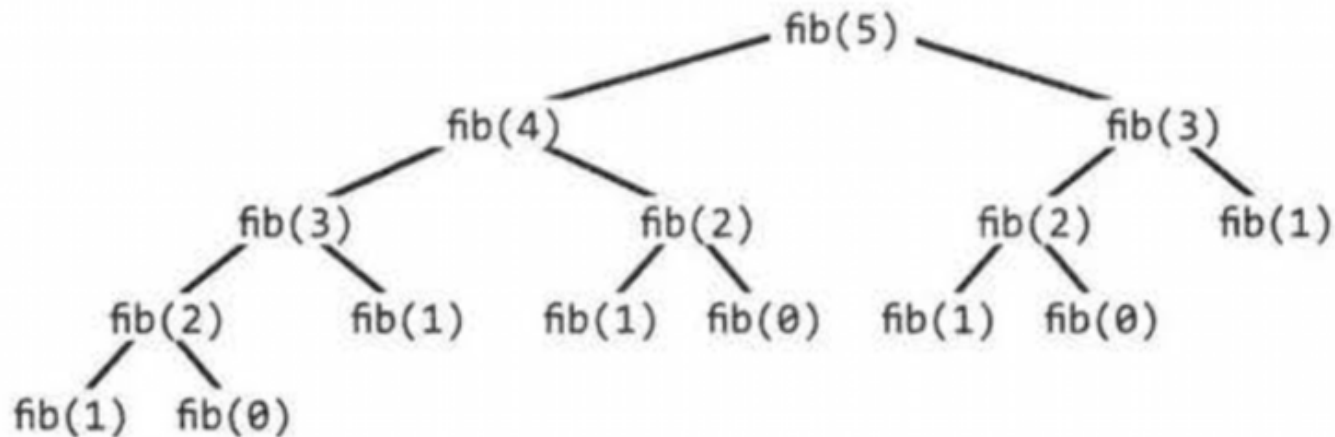
Bài toán Fibonacci với Đệ quy (góc nhìn top-down)

- **Tổng số lượng các đỉnh** trên cây chính là *độ phức tạp về mặt thời gian* của thuật toán.



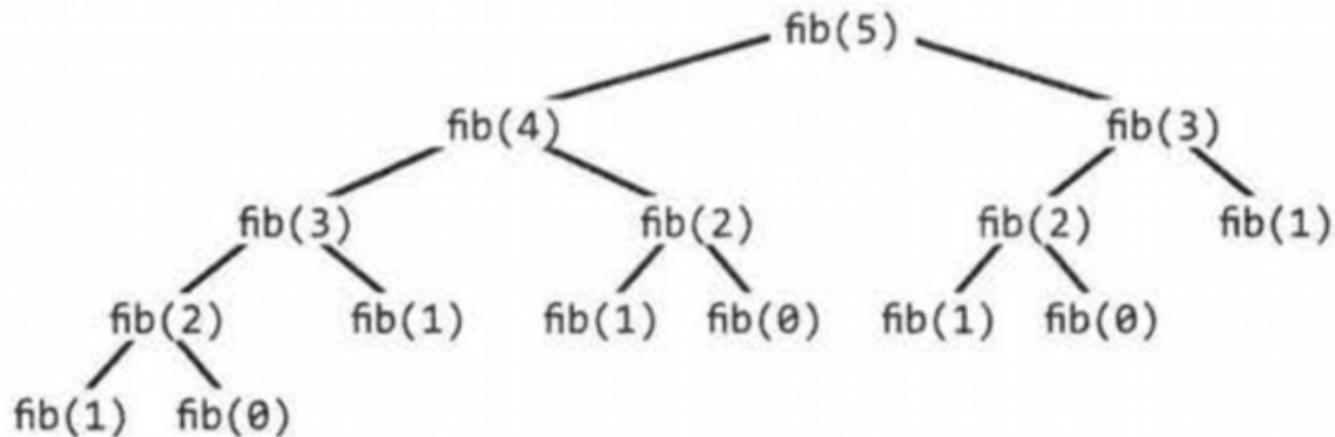
Bài toán Fibonacci với Đệ quy (góc nhìn top-down)

- Tại mỗi đỉnh:
 - Thời gian: $O(1)$
 - Không gian: $O(1)$



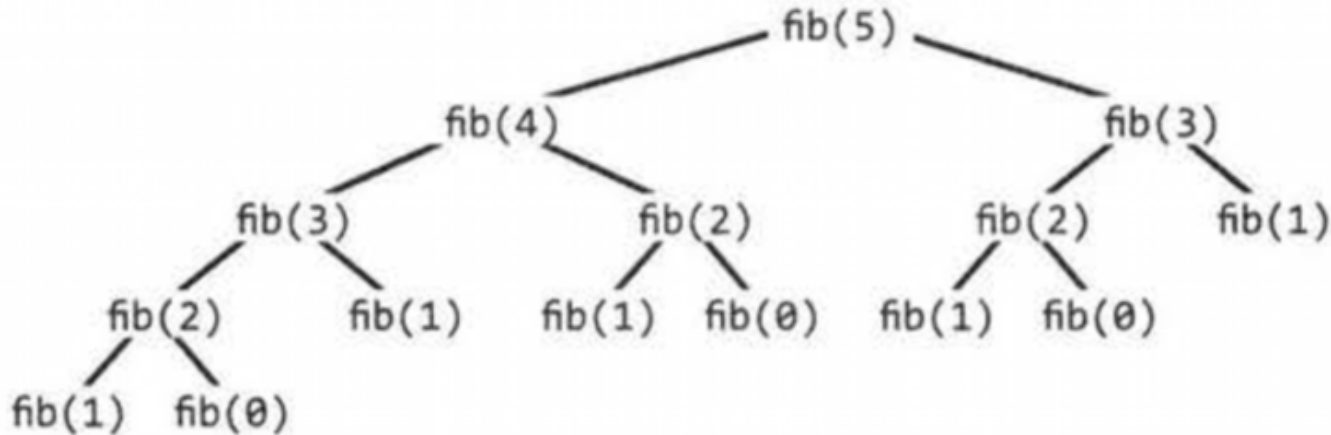
Bài toán Fibonacci với Đệ quy (góc nhìn top-down)

- Tổng thời gian: $O(2^n)$
- Tổng không gian: $O(n)$ - chiều cao của cây, tại sao?



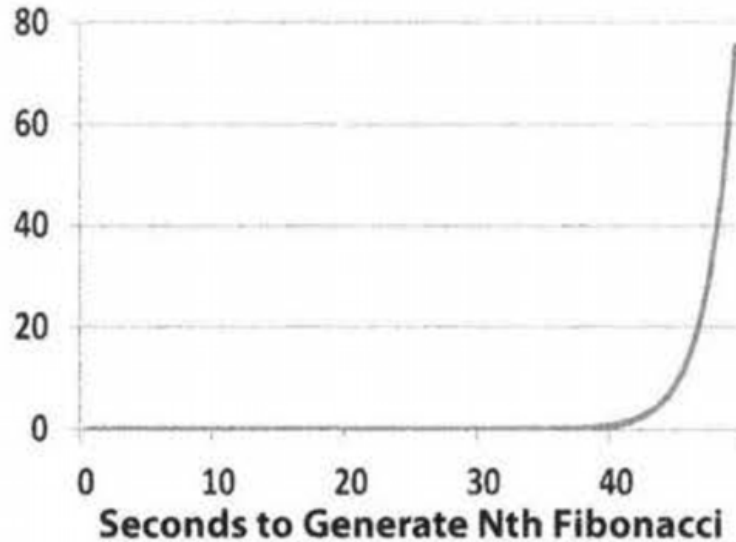
Bài toán Fibonacci với Đệ quy (góc nhìn top-down)

- Tổng không gian: $O(n)$ - chiều cao của cây, tại sao?
- Mỗi lần gọi đệ quy, chương trình lưu lại thông tin của hàm hiện tại vào bộ nhớ stack. Thuật toán gọi đệ quy tối đa n lần, sử dụng $O(n)$ bộ nhớ.



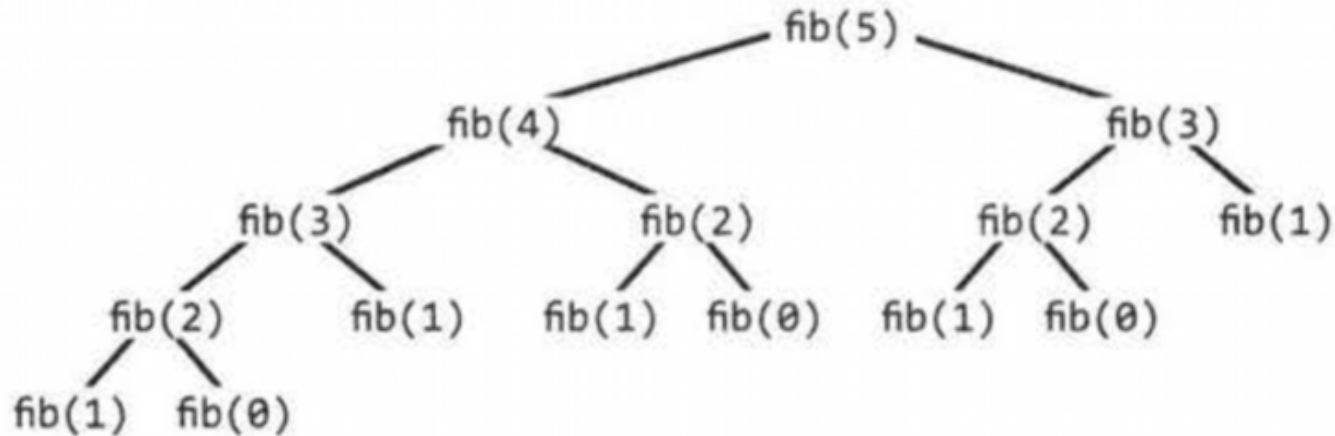
Bài toán Fibonacci với Độ quy (góc nhìn top-down)

-



Bài toán Fibonacci với Độ quy (góc nhìn top-down)

- Chú ý: một số đỉnh trùng nhau trên cây.



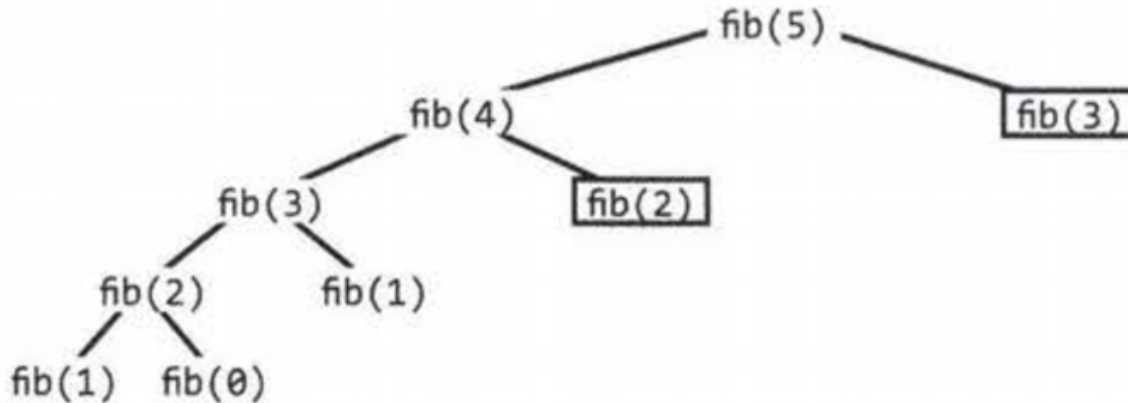
Bài toán Fibonacci với Memoization

```
int fibo(int n) {  
    return fibo(n, new int[n+1])  
}
```

```
int fibo(int i, int[] cache) {  
    if (i==0 || i==1) return i;  
    if (cache[i] == 0) {  
        cache[i] = fibo(i-1, cache) + fibo(i-2, cache)  
    }  
    return cache[i];  
}  
fibo_n = fibo(n);
```

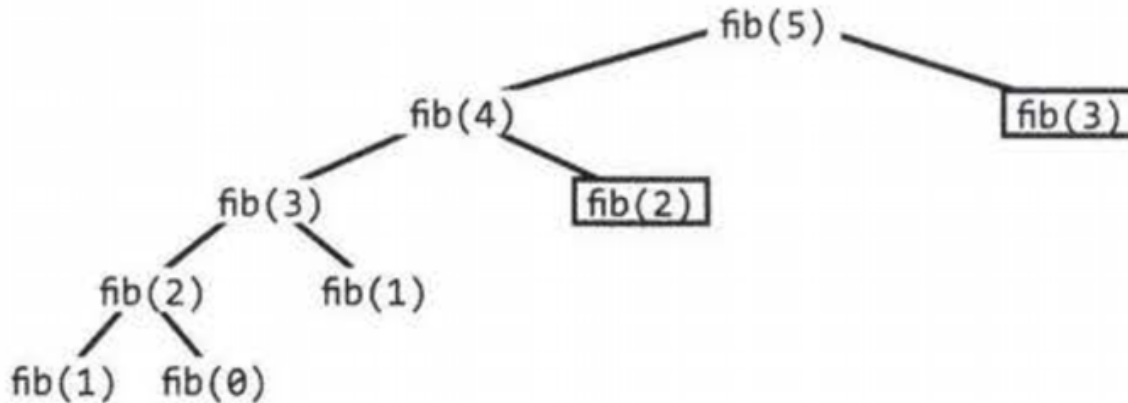
Bài toán Fibonacci với Memoization

- Cây đệ quy chứa ít đỉnh hơn.



Bài toán Fibonacci với Memoization

- Tổng thời gian: $O(n)$
- Tổng không gian: $O(n)$



Bài toán Fibonacci với Quy hoạch động (góc nhìn bottom-up)

```
int fibo(int n) {  
    if (n==0) return 0;  
    else if (n==1) return 1;  
  
    int[] cache = new int[n+1];  
    cache[0] = 0;  
    cache[1] = 1;  
    for (int i=2; i<=n; i++) {  
        cache[i] = cache[i-1] + cache[i-2];  
    }  
    return cache[n];  
}
```

Bài toán Fibonacci với Quy hoạch động (góc nhìn bottom-up)

```
int fibo(int n) {  
    if (n==0) return 0;  
    else if (n==1) return 1;  
  
    int[] cache = new int[n+1];  
    cache[0] = 0;  
    cache[1] = 1;  
    for (int i=2; i<=n; i++) {  
        cache[i] = cache[i-1] + cache[i-2];  
    }  
    return cache[n];  
}
```

- Tổng thời gian: $O(n)$
- Tổng không gian: $O(n)$

Bài toán Fibonacci với Quy hoạch động (góc nhìn bottom-up)

```
int fibo(int n) {  
    if (n==0) return 0;  
    else if (n==1) return 1;  
  
    int[] cache = new int[n+1];  
    cache[0] = 0;  
    cache[1] = 1;  
    for (int i=2; i<=n; i++) {  
        cache[i] = cache[i-1] + cache[i-2];  
    }  
    return cache[n];  
}
```

- Tổng thời gian: $O(n)$
- Tổng không gian: $O(n) \Rightarrow O(1)$

Bài toán Fibonacci với Quy hoạch động (góc nhìn bottom-up)

```
int fibo(int n) {  
    if (n==0) return 0;  
    int a=0;  
    int b=1;  
    for (int i=2; i<=n; i++) {  
        int c = a+b;  
        a = b;  
        b = c;  
    }  
    return a+b;  
}
```

- Tổng thời gian: $O(n)$
- Tổng không gian: $O(1)$

Bài toán Fibonacci với Quy hoạch động (góc nhìn bottom-up)

```
int fibo(int n) {  
    if (n==0) return 0;  
    int a=0;  
    int b=1;  
    for (int i=2; i<=n; i++) {  
        int c = a+b;  
        a = b;  
        b = c;  
    }  
    return a+b;  
}
```

- Tổng thời gian: $O(n) \Rightarrow O(\log^2(n))$?
- Tổng không gian: $O(1)$

Bài toán Fibonacci với phương pháp nhân ma trận (nâng cao)

- Donald E. Knuth đưa ra ma trận đơn vị:

$$Q = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$$

Bài toán Fibonacci với phương pháp nhân ma trận (nâng cao)

- Donald E. Knuth đưa ra ma trận đơn vị:

$$Q = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$$

$$\begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n$$

Bài toán Fibonacci với phương pháp nhân ma trận (nâng cao)

- Donald E. Knuth đưa ra ma trận đơn vị:

$$Q = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$$

$$\begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n$$

$$F_n = Q^{n-1} (1, 1)$$

Bài toán Fibonacci với phương pháp nhân ma trận (nâng cao)

- Donald E. Knuth đưa ra ma trận đơn vị:

$$Q = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$$

$$\begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n$$

$$F_n = Q^{n-1} (1, 1)$$

- Chứng minh: sử dụng **quy nạp**

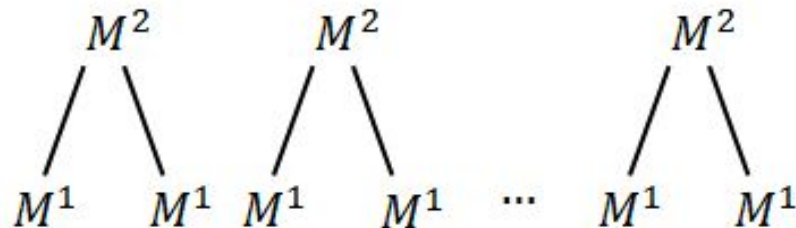
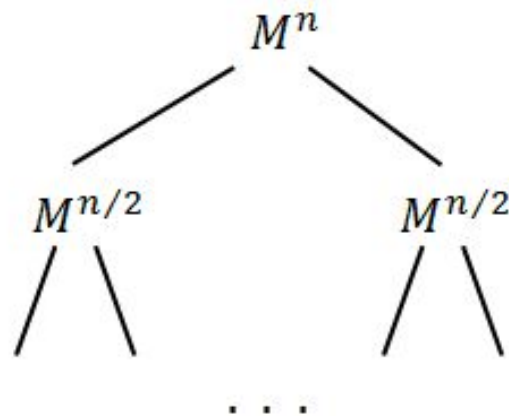
Bài toán Fibonacci với phương pháp nhân ma trận (nâng cao)

$$\begin{bmatrix} 2 & 3 & 4 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1000 \\ 1 & 100 \\ 0 & 10 \end{bmatrix}$$

Bài toán Fibonacci với phương pháp nhân ma trận (nâng cao)

$$\begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n$$

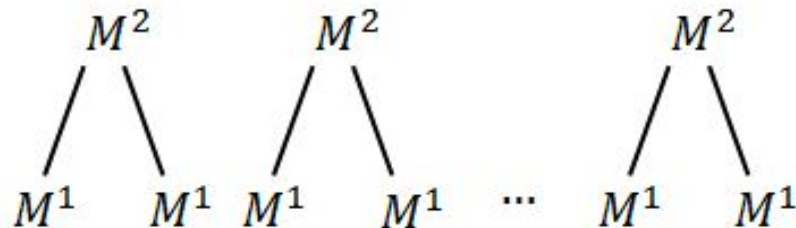
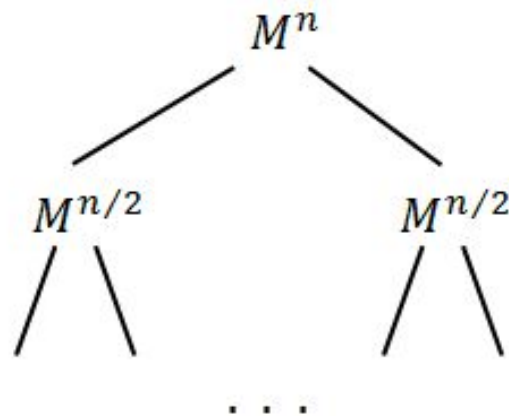
- Tổng thời gian: **$O(\log^2(n))$**
- Tổng không gian: **$O(\log^2(n))$**
- Chạy nhanh khi n rất lớn.



Bài toán Fibonacci với phương pháp nhân ma trận (nâng cao)

$$\begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n$$

- Tổng thời gian: **$O(\log^2(n))$**
- Tổng không gian: **$O(\log^2(n))$**
- Chạy nhanh khi n rất lớn.



Bài tập áp dụng (tổ hợp)

- **Lát gạch 2:** Cho nền nhà có kích thước $3 \times n$. Hãy đếm số cách lát các viên gạch có kích thước 1×3 và 3×1 vào nền nhà trên sao cho toàn bộ nền nhà đều được lát gạch và không có phần nào của các viên gạch nhỏ thừa ra ngoài.
- **Bậc thang:** Một đứa trẻ đứng dưới cầu thang có n bậc thang. Đứa trẻ có thể bước lên 1 bậc, 2 bậc, hoặc 3 bậc một lúc. Hãy đưa ra một thuật toán để đếm số cách khác nhau để đứa trẻ bước lên được bậc thứ n . Đồng thời, phân tích độ phức tạp thuật toán.
- **Lát gạch 3:** Cho nền nhà có kích thước $3 \times n$. Hãy đếm số cách lát các viên gạch có kích thước 1×2 và 2×1 vào nền nhà trên sao cho toàn bộ nền nhà đều được lát gạch và không có phần nào của các viên gạch nhỏ thừa ra ngoài.

Bài tập áp dụng (tổ hợp)

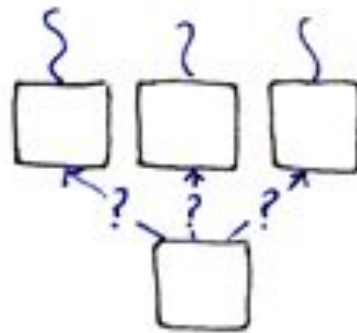
- **Robot:** Một con robot hút bụi được đặt trên một nền gạch có nhiều ô kích thước $m \times n$. Robot bắt đầu công việc hút bụi của mình ở góc trên bên trái nền gạch. Robot chỉ có thể di chuyển theo hướng đông hoặc hướng nam. Biết rằng một số ô gạch có chứa vật cản và robot không thể đi vào những ô này. Hãy đề xuất một thuật toán đếm số cách khác nhau để robot có thể di chuyển đến góc phải dưới của nền gạch. Đồng thời, phân tích độ phức tạp thuật toán.

Bài tập áp dụng (tối ưu)

- **Mua vé:** Có N người đang xếp hàng mua vé. Người thứ i mua vé cho mình thì mất T_i phút. Tuy nhiên, nếu người thứ i mua luôn vé cho cả người đứng sau thì tổng thời gian sẽ mất R_i phút. Tìm tổng thời gian khi những người xếp hàng mua vé một cách tối ưu nhất. Hãy in ra chỉ số của những người cần phải mua vé để đạt được thời gian tối ưu.
- **Hai dãy số:** Cho 2 số nguyên dương m, n và 2 dãy số nguyên $A_1, A_2, \dots, A_m$ và $B_1, B_2, \dots, B_n$. Hãy loại đi một số phần tử của 2 dãy sao cho các số còn lại của 2 dãy (giữ nguyên thứ tự cũ) tạo thành 2 dãy giống nhau và có độ dài k là lớn nhất. In ra k và dãy chung bất kỳ độ dài k .

Bài tập áp dụng (tối ưu)

- **Ma trận:** Cho một ma trận chứa $M \times N$ số nguyên. Một con robot đứng ở lề dưới (biên dưới) của ma trận và mong muốn di chuyển sang lề bên trên (biên trên) của ma trận. Robot có thể chọn vị trí bắt đầu bất kì ở hàng dưới cùng và kết thúc ở bất kỳ ô nào ở hàng trên cùng của ma trận. Khi robot đang ở ô $[i,j]$, nó chỉ có thể di chuyển đến 3 ô $[i-1,j-1]$, $[i-1,j]$, và $[i-1,j+1]$. Hãy tìm tổng giá trị của đường đi nhỏ nhất của robot.



Một số trang web luyện tập

- **Leetcode:** <https://leetcode.com/tag/dynamic-programming/>
- **HackerRank:**
<https://www.hackerrank.com/domains/algorithms?filters%5Bsubdomains%5D%5B%5D=dynamic-programming>